

Information Retrieval

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

What Information Retrieval is

*Information Retrieval (IR) is **finding material** (usually documents) of an **unstructured nature** (usually text) that satisfies an **information need** from within **large collections** (usually stored on computers)*

With the advent of the Web, IR became an important activity for many people for example when they search for information on the **web** or in the **mail box**.

Structured, Unstructured and SemiStructured

From the definition, IR works on **Unstructured Data**.

Unstructured Data is the opposite to the Structured Data which are suitable to be used in traditional Relational Database.

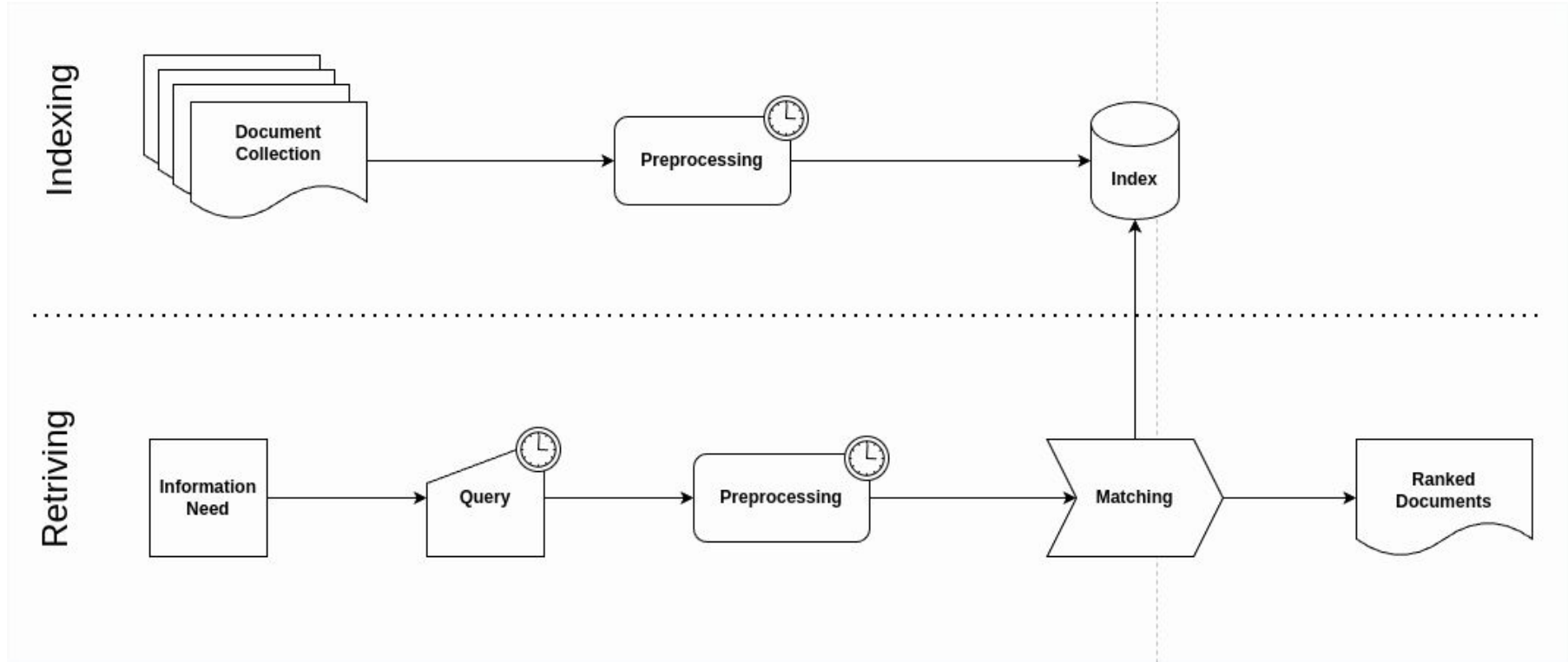
In reality, almost no data are truly “unstructured”. Indeed, if we think about a document, most text has structure, such as headings and paragraphs and footnotes, and this in the web are represented by **tags**.

So, IR facilitates **semi structured** searches and supports further activities on the retrieved documents, such as clustering or classification

What an Information Retrieval System is

An Information Retrieval System aims to provide **documents** belonging to a **collection**, that is **relevant** to an arbitrary **user information need**, which is described by a **query**.

How an Information Retrieval System works



Positive and Negative Results

The **main aim** of a Information Retrieval System is to retrieve all and only those documents that are **relevant**. The **confusion matrix** is a table combining relevant and non relevant documents with those that are retrieved and non retrieved

	Relevant	Non Relevant
Retrieved	<i>True Positive</i>	<i>False Positive</i>
Non Retrieved	<i>False Negative</i>	<i>True Negative</i>

Precision, Recall, Accuracy, F1-Score

An IR system should provide documents that are relevant for the user.

To assess the effectiveness of an IR system we use three indexes:

Precision, Recall, Accuracy, F1-Score

	Relevant	Non Relevant
Retrieved	<i>True Positive</i>	<i>False Positive</i>
Non Retrieved	<i>False Negative</i>	<i>True Negative</i>

$$\text{Precision} = \frac{\sum TP}{\sum TP + FP}$$

$$\text{Recall} = \frac{\sum TP}{\sum TP + FN}$$

$$\text{Accuracy} = \frac{\sum TP + TN}{\sum TP + FP + FN + TN}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision, Recall, Accuracy, F1-Score

Precision: What fraction of the returned results are relevant to the information need?

Recall: What fraction of the relevant documents in the collection were returned by the system?

Accuracy, F1-Score: How many predictions did a model get right? Both range between 0 (all bad predictions) and 1 (all good predictions)

	Relevant	Non Relevant
Retrieved	<i>True Positive</i>	<i>False Positive</i>
Non Retrieved	<i>False Negative</i>	<i>True Negative</i>

$$\text{Precision} = \frac{\sum TP}{\sum TP + FP}$$

$$\text{Recall} = \frac{\sum TP}{\sum TP + FN}$$

$$\text{Accuracy} = \frac{\sum TP + TN}{\sum TP + FP + FN + TN}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved			
Not Retrieved			
TOTAL			

Example 1

An information retrieval system is used to find relevant documents in a **large database of 1000 documents**. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved			
Not Retrieved			
TOTAL			1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system **retrieves 50 documents in total**. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved			50
Not Retrieved			
TOTAL			<i>1000</i>

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that **37 of the retrieved documents are relevant**, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37		50
Not Retrieved			
TOTAL			1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while **15 relevant documents were not retrieved**. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37		50
Not Retrieved	15		
TOTAL			1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37	?	50
Not Retrieved	15	?	?
TOTAL	?	?	1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37	13	50
Not Retrieved	15	935	950
TOTAL	52	948	1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37	13	50
Not Retrieved	15	935	950
TOTAL	52	948	1000

$$TP = 37$$

$$TN = 935$$

$$FP = 13$$

$$FN = 15$$

Example 1

- Precision = $TP / (TP + FP) = 37 / (37+13) = 0.74$
- Recall = $TP / (TP + FN) = 37 / (37 + 15) \approx 0.71$
- Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (37+935)/1000 \approx 0.97$
- F1-score = $2*Precision*Recall/(Precision + Recall) = 2*(0.74 * 0.71)/(0.74 + 0.71) \approx 0.73$

	Relevant	Non Relevant	TOTAL
Retrieved	37	13	50
Not Retrieved	15	935	950
TOTAL	52	948	1000

TP = 37

TN = 935

FP = 13

FN = 15

Example 2

A bank uses a model to predict if a loan applicant will default on their loan. From a sample of 2000 applications, the model's performance is as follows: 1500 applicants who did not default were correctly identified, and 150 who did default were also correctly identified. However, the model incorrectly predicted that 200 applicants would default when they didn't, and it failed to identify 150 applicants who actually defaulted. Calculate the precision, recall, accuracy, and F1 score.

True Positive	150
True Negative	1500
False Positive	200
False Negative	150

	Relevant	Non Relevant	TOTAL
Retrieved	150	200	350
Not Retrieved	150	1500	1650
TOTAL	300	1700	2000

Example 2

- Precision = $TP / (TP + FP) = 150 / (150 + 200) \approx 0.43$
- Recall = $TP / (TP + FN) = 150 / (150 + 150) = 0.50$
- Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (1650) / 2000 \approx 0.83$
- F1-score = $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}) = 2 * (0.43 * 0.50) / (0.43 + 0.50) \approx 0.46$

True Positive	150
True Negative	1500
False Positive	200
False Negative	150

	Relevant	Non Relevant	TOTAL
Retrieved	150	200	350
Not Retrieved	150	1500	1650
TOTAL	300	1700	2000

What does relevance mean?

Relevance is the degree to which a query response satisfies a user who is searching for information.

It depends on the the context in which the query was performed.

A single search application may be used in different contexts by users with **different needs and expectations**

What does relevance mean?

Relevant documents are subjective:

For example:

Suppose we have a search engine of **climate data**.

- a **university researcher** is interested in studying long-term climate trend
- a **farmer** is interested in calculating the likely date of the last frost of spring
- a **civil engineer** is interested in rainfall patterns and the frequency of floods
- a **student** is interested in planning a vacation to a region

The simplest Information Retrieval System

Suppose we have all the Shakespeare's Works.

We want to find all documents containing the words

Caesar and Brutus but not Calpurnia.

The simplest form of document retrieval is for a computer to do this sort of **linear scan** through documents.

This process is known as **grepping**. Remind the **Unix grep command**.

Information retrieval problems

- When we have a small document collection (like all Shakespeare's works, containing 38 plays with less than one million words of text in total) this is not a big deal, but having billions or trillions of documents, search time becomes important.
- Sometimes we would like to search for documents using not an exact pattern, but for example “***search for the word Romans near to countrymen***” (where near means “within 5 words”)
- In many case there are many documents satisfying the query, and we would like to have **a rank**

Linear vs Indexed Search

Unix **grep command** works linearly (and this is the reason of its slowness).

The alternative Search is an **Indexed Search**, which makes use of Index.

Indexing documents means building a **term-document incidence matrix**.

Zeros and Ones indicate whether a term is found in the document

		Documents					
		Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth ..
Terms	Antony	1	1	0	0	0	1
	Brutus	1	1	0	1	0	0
	Caesar	1	1	0	1	1	1
	Calpurnia	0	1	0	0	0	0
	Cleopatra	1	0	0	0	0	0
	mercy	1	0	1	1	1	1
	worser	1	0	1	1	1	0
...							

1 = term is contained in the document
0 = term is NOT contained in the document

Boolean Retrieval Model

By looking at the term-document matrix it will be easy to find all documents containing one or more terms.

In the query, terms are combined with operators **AND**, **OR** and **NOT**.

This retrieval is named **Boolean Retrieval Model**

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	..
Terms	1	1	0	0	0	1	
Antony	1	1	0	1	0	0	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

1 = term is contained in the document
0 = term is NOT contained in the document

The big problem of the term-doc matrix

Let's suppose we have a collection of 1 million documents.

Suppose each document is about 1000 words (3 pages)

Suppose all documents have 500.000 distinct terms

The size of the term-document incidence matrix is: $10^6 * 5 * 10^5 = 5 * 10^{11}$ bit,
more than 465Gb (approx 58GB):

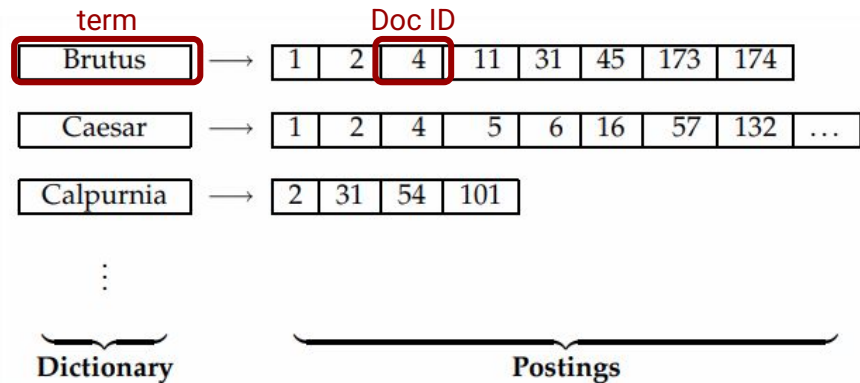
- very big to keep it in memory
- the matrix is sparse (99.8% of cells are zero)

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

The solution: inverted index

This is the major concept in Information Retrieval: **the inverted index**

We have a **dictionary** of terms, for each term we have a list that records which documents the term occurs in. This is named **postings** or **postings list**



Dictionary vs Vocabulary

Dictionary and Vocabulary are often used as synonyms

Here instead we use a convention:

- **Dictionary**: is the data structure containing words
- **Vocabulary**: is the set of words

How to build an inverted index

To gain benefit of indexing at retrieval time, we have to **build the index in advance**:

1. Collect the documents to be indexed

Friends, Romans, countrymen. So let it be with Caesar ...

2. Tokenize the text, turning each document into a list of tokens

Friends Romans countrymen So ...

How to build an inverted index

To gain benefit of indexing at retrieval time, we have **to build the index in advance**:

3. Do linguistic preprocessing, producing a list of **normalized tokens**, which are the indexing **terms**

friend	roman	countryman	so	...
--------	-------	------------	----	-----

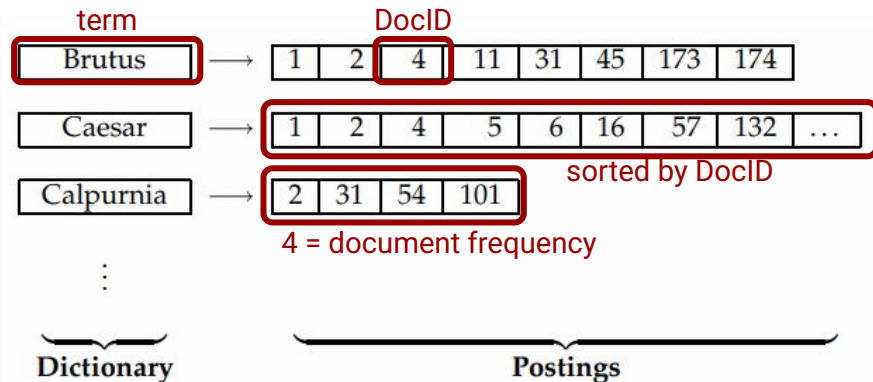
4. Index the documents that each term occurs in by creating an **inverted index**, consisting of a **dictionary and postings**.

How to build an inverted index

During inverted index creation,
documents are identified univocally
by an ID (docID)

Terms in the dictionary and DocID in
the postings are **sorted** and merged

The dictionary contains also some
statistics, for example the **document**
frequency = the length of a posting



An example

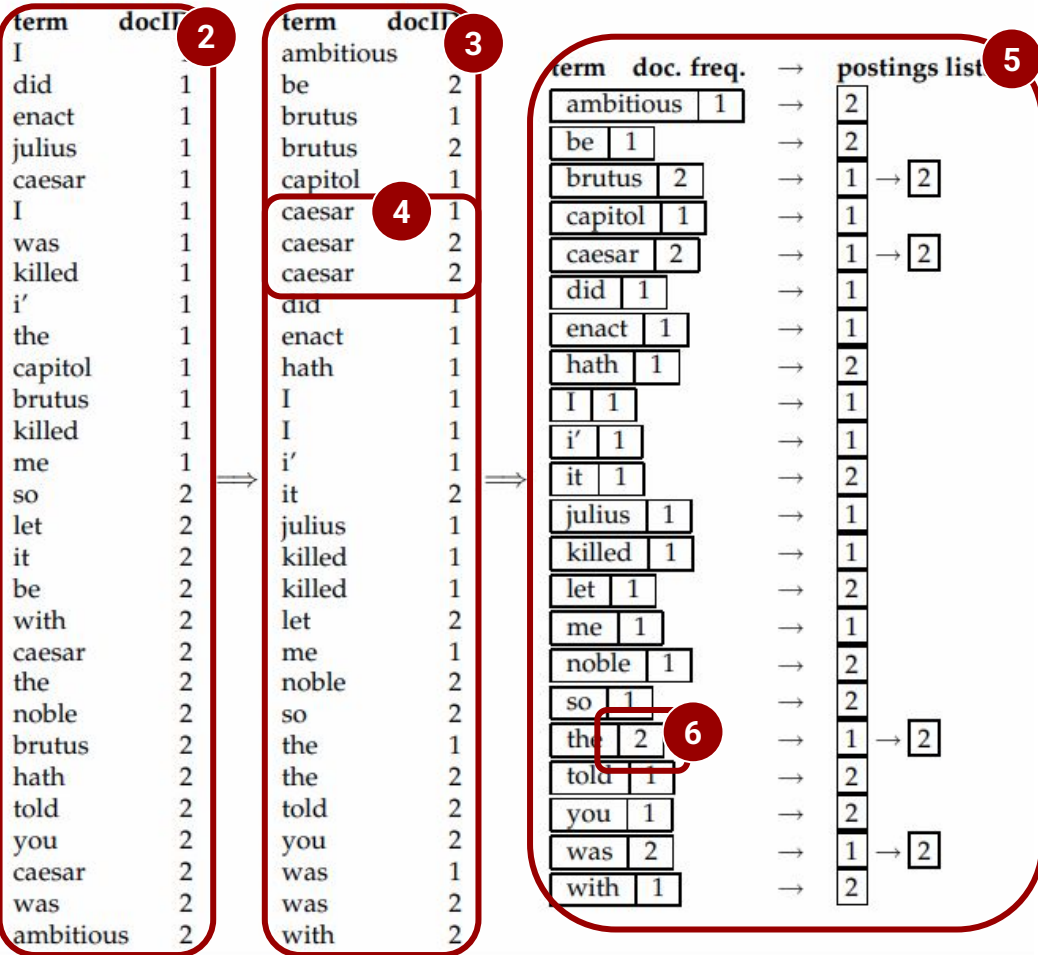
1. Collect all documents
2. Terms are extracted, tokenized, preprocessed and tagged with the DocID
3. Terms are sorted alphabetically
4. Instances of the same term are then grouped by word and then by documentID
5. For each unique term, postings list is built
6. Some information (such the document frequency) are stored (for improving query efficiency or for **ranking results**)

Doc 1

I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:



Challenges for inverted index

- Which data structure is the best for managing Dictionary and Postings?
 - Single Linked List
 - Variable length array
 - ...
- Where should I store both Dictionary and Postings?
 - In Memory
 - In Disk

We will discuss about this details which **affect the performance** of a IR system.

Processing Boolean Conjunctive Queries

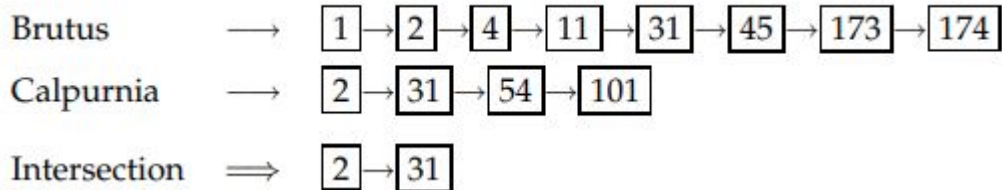
How a Query can work on Boolean Retrieval System?

Suppose we want to process this **simple conjunctive query**:

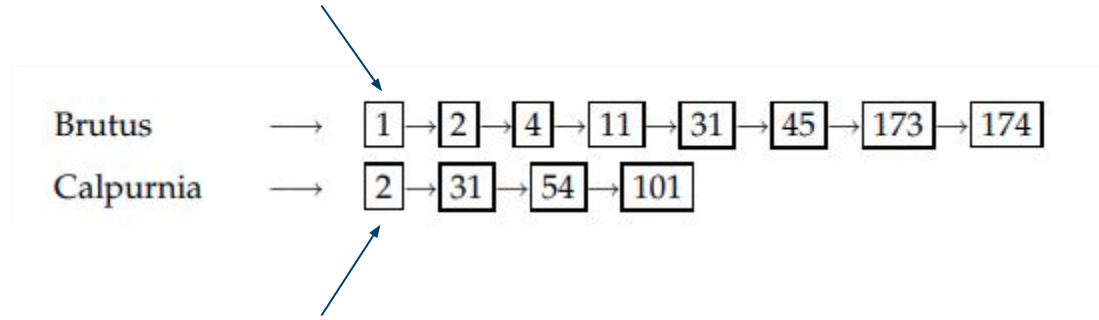
Brutus AND Calpurnia

What we need to do?

1. Locate Brutus on Dictionary
2. Retrieve its postings
3. Locate Calpurnia on Dictionary
4. Retrieve its postings
5. Intersect the two postings lists



Implementing Intersection



We maintain **pointers** into both lists and walk through the two postings lists simultaneously

Implementing Intersection

Computing the intersection between two Postings **must be efficient**.

Here is a well known efficient algorithm

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

Implementing Intersection: complexity

If x and y are the lengths of the postings p_1 and p_2 , the complexity of the algorithm is $O(x + y) = O(N)$ where N is the number of documents in the collection.

So the **algorithm is linear**.

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

Processing Boolean Disjunctive Queries

How a Query can work on Boolean Retrieval System?

Suppose we want to process this **simple disjunctive query**:

Brutus OR Calpurnia

What we need to do?

1. Locate Brutus on Dictionary
2. Retrieve its postings
3. Locate Calpurnia on Dictionary
4. Retrieve its postings
5. Merge the two postings lists

Brutus

→ 1 → 2 → 4 → 11 → 31 → 45 → 173 → 174

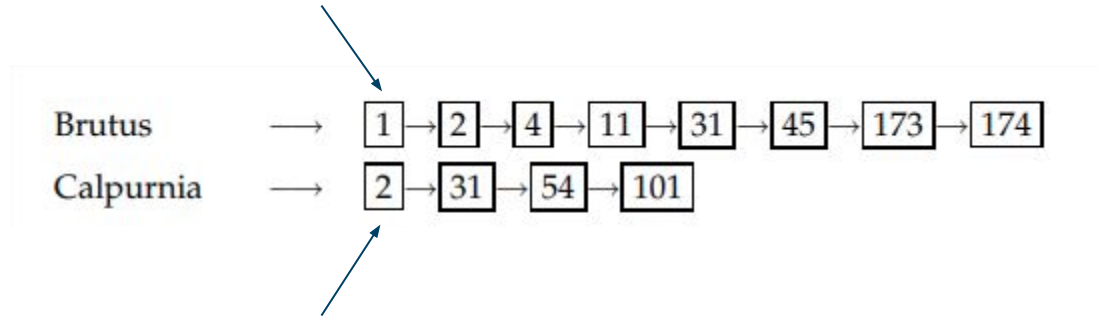
Calpurnia

→ 2 → 31 → 54 → 101

Union

→ 1 2 4 11 31 45 54 101 173 174

Implementing Union



We maintain **pointers** into both lists and walk through the two postings lists simultaneously

Implementing Union

Computing union between two
Postings is very easy and efficient.

Here is a well known efficient algorithm

```
1 union(p1, p2)
2 {
3     answer = NULL;
4     while (p1 <> NULL and p2 <> NULL)
5     {
6         if (docId(p1) == docId(p2))
7         {
8             add(answer, p1);
9             p1 = next(p1);
10            p2 = next(p2);
11        }
12        else if (docId(p1) < docId(p2))
13        {
14            add(answer, p1);
15            p1 = next(p1);
16        }
17        else
18        {
19            add(answer, p2);
20            p2 = next(p2);
21        }
22    }
23    return answer;
24 }
```


Implementing Union: complexity

If x and y are the lengths of the postings p_1 and p_2 , the complexity of the algorithm is $O(x + y) = O(N)$ where N is the number of documents in the collection.

So the **algorithm is linear**.

```
1 union(p1, p2)
2 {
3     answer = NULL;
4     while (p1 <> NULL and p2 <> NULL)
5     {
6         if (docId(p1) == docId(p2))
7         {
8             add(answer, p1);
9             p1 = next(p1);
10            p2 = next(p2);
11        }
12        else if (docId(p1) < docId(p2))
13        {
14            add(answer, p1);
15            p1 = next(p1);
16        }
17        else
18        {
19            add(answer, p2);
20            p2 = next(p2);
21        }
22    }
23    return answer;
24 }
```

Query Optimization

Sometimes, queries are complex and involves many terms, such as:

Brutus AND Caesar AND Calpurnia

Query optimization is the process to organize the work of answering a query so that the least total amount of work needs is done by the system.

An heuristic is to process terms in order of **increasing document frequency**

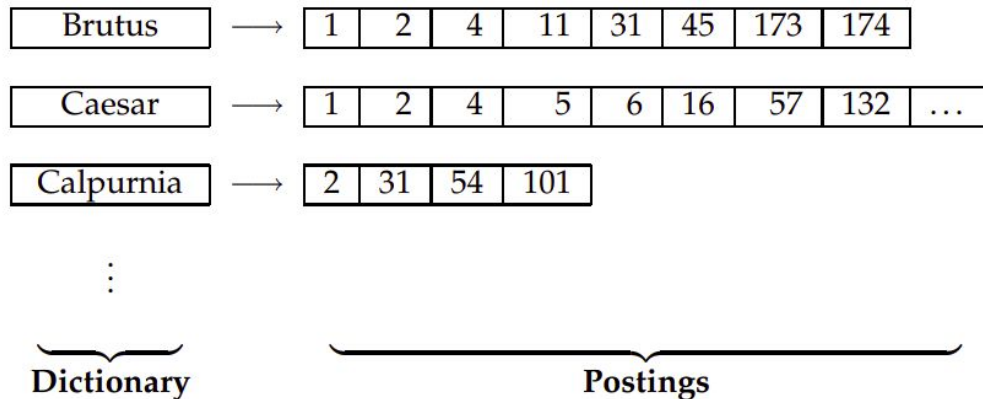
Query Optimization

So, for example, supposing that the the document frequency is the one showed, the query

Brutus AND Caesar AND Calpurnia

becomes

(Calpurnia AND Brutus) AND Caesar



Optimized Intersection

Line 1: compute a sorting of the terms by increasing document frequency

Line 5: make an intersection between two posting list (the current intermediate result and the posting list of the next term having lower document frequency)

```
INTERSECT( $\langle t_1, \dots, t_n \rangle$ )  
1  terms  $\leftarrow$  SORTBYINCREASINGFREQUENCY( $\langle t_1, \dots, t_n \rangle$ )  
2  result  $\leftarrow$  postings(first(terms))  
3  terms  $\leftarrow$  rest(terms)  
4  while terms  $\neq$  NIL and result  $\neq$  NIL  
5  do result  $\leftarrow$  INTERSECT(result, postings(first(terms)))  
6    terms  $\leftarrow$  rest(terms)  
7  return result
```

Optimized Intersection

Notice that:

- *result* is a variable always kept in memory
- *postings(first(terms))* is a function that reads from disk

```
INTERSECT( $\langle t_1, \dots, t_n \rangle$ )  
1  terms  $\leftarrow$  SORTBYINCREASINGFREQUENCY( $\langle t_1, \dots, t_n \rangle$ )  
2  result  $\leftarrow$  postings(first(terms))  
3  terms  $\leftarrow$  rest(terms)  
4  while terms  $\neq$  NIL and result  $\neq$  NIL  
5  do result  $\leftarrow$  INTERSECT(result, postings(first(terms)))  
6    terms  $\leftarrow$  rest(terms)  
7  return result
```

More complex queries

Some query can be more complex, like this

(madding OR crowd) AND (ignoble OR strife) AND (killed OR slain)

we will get the frequencies for all terms, and we can then (conservatively) estimate the size of each **OR** by the **sum of the frequencies of its disjuncts**.

We can then process the query in increasing order of the size of each disjunctive term.

However, in many circumstances, a query is **purely conjunctive**

Preferred IR features

Information Retrieval having only Boolean Operations (AND, OR, NOT) **are very limited**. Often we would like to have other features

- **Proximity operators**: a way to specify how close or far two terms must appear into the documents
- **Error Tolerance**: prevent or adjust **mistakes** and inconsistent words
- **Compounds and Phrases**: allow using **phrases** that denote a concept such as “operating system”
- **Statistics**: retrieving documents depends on the **term frequency**
- **Ranking**: the returned documents should be sorted by rank